

C PROGRAMMING

Unit-IV: Functions & Pointers

Comprehensive Study Notes

Topic 1: Introduction to Functions

1.1 What is a Function?

A **function** is a self-contained block of code that performs a specific task. Functions help in *modular programming* — breaking large programs into smaller, manageable pieces.

1.2 Types of Functions in C

Type	Description	Example
Library Functions	Pre-defined functions in C header files	<code>printf()</code> , <code>scanf()</code> , <code>sqrt()</code>
User-defined Functions	Functions created by programmers	<code>int sum(int a, int b)</code>

1.3 Advantages of Functions

- **Reusability:** Write once, use multiple times
- **Modularity:** Divide complex problems into smaller tasks
- **Readability:** Easy to understand and maintain
- **Debugging:** Easier to find and fix errors
- **Code sharing:** Functions can be used across programs

Topic 2: User-defined Functions

2.1 Elements of a Function

Element	Description
Function Declaration	Also called <i>prototype</i> — tells compiler about the function

Function Definition	Body of the function with actual code
Function Call	Invoking the function in the program

2.2 Function Syntax

```
return_type function_name(parameter_list)
{
    // Body of function
    return value;
}
```

2.3 Program: Simple Function

```
#include <stdio.h>

// Function declaration (prototype)
void greet();

int main()
{
    greet(); // Function call
    greet(); // Function call again
    return 0;
}

// Function definition
void greet()
{
    printf("Hello! Welcome to C Programming.\n");
}
```

Output:

```
Hello! Welcome to C Programming.
Hello! Welcome to C Programming.
```

Topic 3: Types of User-defined Functions

3.1 Based on Arguments and Return Value

Type	Arguments	Return Value	Example
Type 1	No	No	void show()
Type 2	No	Yes	int getValue()
Type 3	Yes	No	void display(int x)
Type 4	Yes	Yes	int add(int a, int b)

3.2 Type 1: No Arguments, No Return

```
#include <stdio.h>

void printLine();

int main()
{
    printLine();
    return 0;
}

void printLine()
{
    printf("-----\n");
}
```

3.3 Type 2: No Arguments, Returns Value

```
#include <stdio.h>

int getNumber();
```

```
int main()
{
    int num = getNumber();
    printf("Number: %d\n", num);
    return 0;
}

int getNumber()
{
    return 42;
}
```

3.4 Type 3: Arguments, No Return

```
#include <stdio.h>

void printSquare(int n);

int main()
{
    int x = 5;
    printSquare(x); // 5 passed as argument
    return 0;
}

void printSquare(int n)
{
    printf("Square of %d = %d\n", n, n * n);
}
```

3.5 Type 4: Arguments with Return Value

```
#include <stdio.h>

int add(int a, int b);

int main()
{
    int result = add(10, 20);
    printf("Sum = %d\n", result);
}
```

```
        return 0;  
    }  
  
    int add(int a, int b)  
    {  
        return a + b;  
    }
```

Output:

Sum = 30

Topic 4: Parameter Passing Methods

4.1 Call by Value

In **call by value**, a copy of the actual argument is passed to the function. Changes made inside the function do NOT affect the original variable.

```
#include <stdio.h>

void change(int x);

int main()
{
    int a = 10;
    printf("Before: a = %d\n", a); // 10
    change(a);
    printf("After: a = %d\n", a);  // 10 (unchanged)
    return 0;
}

void change(int x)
{
    x = x + 5;
    printf("Inside: x = %d\n", x); // 15
}
```

Output:

```
Before: a = 10
Inside: x = 15
After: a = 10
```

4.2 Call by Reference (using Pointers)

In **call by reference**, the address of the actual argument is passed. Changes inside the function DO affect the original variable.

```
#include <stdio.h>

void change(int *x);

int main()
{
    int a = 10;
    printf("Before: a = %d\n", a);    // 10
    change(&a);
    printf("After: a = %d\n", a);     // 15 (changed)
    return 0;
}

void change(int *x)
{
    *x = *x + 5;
    printf("Inside: *x = %d\n", *x); // 15
}
```

Output:

```
Before: a = 10
Inside: *x = 15
After: a = 15
```

4.3 Comparison

Feature	Call by Value	Call by Reference
What is passed	Copy of value	Address of variable
Original modified?	No	Yes
Memory usage	More (copy created)	Less (no copy)
Speed	Slower for large data	Faster for large data

Topic 5: Storage Classes

5.1 Storage Classes Table

Storage Class	Keyword	Scope	Lifetime	Initial Value
Automatic	auto	Local to block	Until block ends	Garbage
Register	register	Local to block	Until block ends	Garbage
Static	static	Local to block	Entire program	0
External	extern	Global	Entire program	0

5.2 auto Storage Class

Default storage class for local variables.

```
#include <stdio.h>

int main()
{
    auto int a = 10; // auto is optional
    printf("a = %d\n", a);
    return 0;
}
```

5.3 register Storage Class

Suggests the variable to be stored in a CPU register for faster access.

```
#include <stdio.h>

int main()
{
    register int counter;
```

```
    for (counter = 1; counter <= 5; counter++)  
    {  
        printf("%d ", counter);  
    }  
    return 0;  
}
```

5.4 static Storage Class

Static variables retain their value between function calls.

```
#include <stdio.h>  
  
void counter()  
{  
    static int count = 0; // Initialized only once  
    count++;  
    printf("Count = %d\n", count);  
}  
  
int main()  
{  
    counter(); // Count = 1  
    counter(); // Count = 2  
    counter(); // Count = 3  
    return 0;  
}
```

Output:

```
Count = 1  
Count = 2  
Count = 3
```

5.5 extern Storage Class

Used to access global variables defined in another file or location.

```
#include <stdio.h>

int globalVar = 100; // Global variable

void display()
{
    extern int globalVar;
    printf("Global: %d\n", globalVar);
}

int main()
{
    display();
    return 0;
}
```

Topic 6: Recursion

6.1 What is Recursion?

Recursion is a function calling itself to solve a problem by breaking it into smaller subproblems.

6.2 Recursion Flow Diagram

```
factorial(5)
|
+--> 5 * factorial(4)
      |
      +--> 4 * factorial(3)
            |
            +--> 3 * factorial(2)
                  |
                  +--> 2 * factorial(1)
                        |
                        +--> 1 (base case)
                              |
                              <-- 2 * 1 = 2
                                    |
                                    <-- 3 * 2 = 6
                                          |
                                          <-- 4 * 6 = 24
                                                |
                                                <-- 5 * 24 = 120
```

6.3 Program: Factorial using Recursion

```
#include <stdio.h>

int factorial(int n);

int main()
{
    int num = 5;
    printf("Factorial of %d = %d\n", num, factorial(num));
    return 0;
}

int factorial(int n)
{
```

```
    if (n == 0 || n == 1)
        return 1; // Base case
    else
        return n * factorial(n - 1); // Recursive call
}
```

Output:

Factorial of 5 = 120

6.4 Program: Fibonacci Series using Recursion

```
#include <stdio.h>

int fibonacci(int n);

int main()
{
    int n = 10, i;
    printf("Fibonacci series: ");
    for (i = 0; i < n; i++)
        printf("%d ", fibonacci(i));
    printf("\n");
    return 0;
}

int fibonacci(int n)
{
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Output:

Fibonacci series: 0 1 1 2 3 5 8 13 21 34

6.5 Recursion vs Iteration

Recursion**Iteration**

Function calls itself	Uses loops (for, while)
Uses stack memory	Uses less memory
Slower (overhead of function calls)	Faster
Infinite recursion causes stack overflow	Infinite loop may hang program

Topic 7: Pointers

7.1 What is a Pointer?

A **pointer** is a variable that stores the *memory address* of another variable. Pointers are one of the most powerful features of C.

7.2 Pointer Declaration and Initialization

```
int a = 10;
int *ptr;    // Declaration of pointer
ptr = &a;    // ptr now stores address of a

printf("Value of a: %d\n", a);        // 10
printf("Address of a: %p\n", &a);    // e.g., 0x7ffeef
printf("Value of ptr: %p\n", ptr);   // e.g., 0x7ffeef (same as &a)
printf("Value at ptr: %d\n", *ptr);  // 10 (dereferencing)
```

7.3 Pointer Operators

Operator	Name	Meaning
&	Address-of	Returns address of a variable
*	Dereference	Returns value at the address

7.4 Program: Basic Pointer Operations

```
#include <stdio.h>

int main()
{
    int a = 10;
    int *ptr = &a;
```

```
printf("Initial value: %d\n", a);

*ptr = 20; // Change value using pointer

printf("After *ptr = 20: %d\n", a); // 20

return 0;
}
```


Topic 8: Pointer Arithmetic

8.1 Pointer Arithmetic Operations

Operation	Meaning
ptr++	Move to next memory location (4 bytes ahead for int*)
ptr--	Move to previous memory location
ptr + n	Move n locations ahead
ptr - n	Move n locations back
ptr1 - ptr2	Number of elements between two pointers

8.2 Program: Pointer Arithmetic with Array

```
#include <stdio.h>

int main()
{
    int arr[] = {10, 20, 30, 40, 50};
    int *ptr = arr; // Points to arr[0]

    printf("arr[0] = %d\n", *ptr);    // 10
    ptr++;
    printf("arr[1] = %d\n", *ptr);    // 20
    ptr = ptr + 2;
    printf("arr[3] = %d\n", *ptr);    // 40

    // Traversing array with pointer
    ptr = arr;
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", *(ptr + i));
    }
    // Output: 10 20 30 40 50
}
```

```
    return 0;
}
```

Topic 9: Pointers and Arrays

9.1 Array Name as Pointer

The array name is a pointer to the first element. `arr` is equivalent to `&arr[0]`.

9.2 Program: Array Traversal using Pointers

```
#include <stdio.h>

int main()
{
    int arr[] = {2, 4, 6, 8, 10};
    int *ptr = arr;
    int sum = 0;

    for (int i = 0; i < 5; i++)
    {
        sum += *(ptr + i);
    }

    printf("Sum = %d\n", sum); // 30
    printf("Average = %.1f\n", sum / 5.0); // 6.0

    return 0;
}
```

9.3 Array Access Methods Comparison

Method	To access arr[i]
Array subscript	arr[i]

Pointer arithmetic	<code>*(arr + i)</code>
Using pointer variable	<code>*(ptr + i)</code>

Topic 10: Pointers and Strings

10.1 String using Pointer

```
#include <stdio.h>

int main()
{
    char *str = "Hello World";

    printf("String: %s\n", str);

    // Printing character by character
    while (*str != '\0')
    {
        printf("%c ", *str);
        str++;
    }

    return 0;
}
```

10.2 Program: String Length using Pointer

```
#include <stdio.h>

int stringLength(char *str)
{
    int count = 0;
    while (*str != '\0')
    {
        count++;
        str++;
    }
    return count;
}

int main()
```

```
{  
    char str[] = "Programming";  
    printf("Length of '%s' = %d\n", str, strlen(str));  
    return 0;  
}
```

Output:

```
Length of 'Programming' = 11
```

Topic 11: Pointer to Pointer

11.1 Concept

A **pointer to pointer** (double pointer) stores the address of another pointer.

11.2 Program: Double Pointer

```
#include <stdio.h>  
  
int main()  
{  
    int a = 10;  
    int *ptr = &a;  
    int **pptr = &ptr;  
  
    printf("a = %d\n", a);           // 10  
    printf("*ptr = %d\n", *ptr);      // 10  
    printf("***pptr = %d\n", **pptr); // 10  
  
    **pptr = 30;  
    printf("After **pptr = 30:\n");  
    printf("a = %d\n", a);           // 30  
  
    return 0;  
}
```

Topic 12: Command Line Arguments

12.1 What are Command Line Arguments?

Values passed to a C program when it is executed from the command line. They are received by `main()` function parameters `argc` and `argv`.

- `argc` : argument count (number of arguments)
- `argv` : argument vector (array of strings)

12.2 Program: Command Line Arguments

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Number of arguments: %d\n", argc);

    for (int i = 0; i < argc; i++)
    {
        printf("argv[%d] = %s\n", i, argv[i]);
    }

    return 0;
}
```

Execution:

```
C:\> program.exe Hello World 123
Number of arguments: 4
argv[0] = program.exe
argv[1] = Hello
argv[2] = World
argv[3] = 123
```

Topic 13: HPU Important Questions

13.1 Short Answer Questions (2-5 Marks)

1. What is a function? Explain its advantages.
2. Differentiate between call by value and call by reference.
3. What is recursion? Write recursive factorial function.
4. What is a pointer? Explain pointer declaration.
5. Explain pointer arithmetic with example.
6. What are storage classes? Name them.
7. Difference between static and auto storage class.
8. What is command line argument? Explain argc and argv.

13.2 Long Answer Questions (10 Marks)

1. Explain types of user-defined functions with examples.
2. Explain call by value and call by reference with examples.
3. Explain recursion with factorial and Fibonacci examples.
4. Explain pointers and pointer arithmetic in detail.
5. Explain storage classes in C with examples.
6. Explain relationship between pointers and arrays.

13.3 Program Writing Questions (5 Marks)

1. Write a program to find factorial using recursion.
2. Write a program to swap two numbers using pointers.
3. Write a program to find length of a string using pointer.
4. Write a program to demonstrate static storage class.
5. Write a program to add two numbers using function.

Objective Questions (1 Mark)

1. Which keyword is used to declare a pointer? (a) ptr (b) * (c) & (d) pointer **Answer: (b)**

2. Operator used to get address of variable: (a) * (b) & (c) @ (d) # **Answer: (b)**
3. Default storage class in C: (a) static (b) register (c) auto (d) extern **Answer: (c)**
4. Which function calling method can modify actual arguments? (a) Call by value (b) Call by reference (c) Recursion (d) None **Answer: (b)**
5. Which contains address of another pointer? (a) Single pointer (b) Double pointer (c) Array (d) Structure **Answer: (b)**

End of Unit-IV Notes